Team: Keyboard Gremlins

COS301 Capstone Project
University of Pretoria

Hackathon Platform

API Service Contract

Name	Student Number
Ayush Sanjith	23535424
Jasveer Ramdhaw	23537036
Rene Reddy	23558572
Heinrich Römer	23538181
Kwaku Asiedu (Sam)	18100156

Team Contact: keyboardgremlins@gmail.com

Team Creation and Management **3**
Service Name: Create Team Service 3
Service Name: Get My Team For Event Service 4
Service Name: Request to Join Team Service 6
Service Name: View Join Requests Service 7
Service Name: Request To Join Team By Code Service 8
Service Name: Approve or Reject Join Request Service 9
Service Name: Leave Team Service 10
Service Name: View Team Members Service 11
File Storage - Specs and Submission Upload/Download **12**
Service Name: Upload Level File Service 12
Service Name: Get Level File Download URL Service 13
Service Name: List Level Files Service 14
Service Name: Delete Level File Service 15
Service Name: Upload Solver File Service 16
Service Name: Upload Problem Statement Service 22
Service Name: Get Problem Statement Download URL Service 23
Service Name: Upload Submission Service 24
Service Name: Get Submission Output Download URL Service 25
Service Name: Get Source Archive Download URL Service 26
Service Name: Get Scoring Log Download URL Service (Single Submission) 27
Admin Event Management **28**
Service Name: Create Event Service 28
Service Name: Get Admin Events Service 30
Service Name: Update Event Service 31
Service Name: Patch Event Status Service 32
Service Name: Get Event Status Service 33
Authorization **34**
Service Name: Post Register User 34
Service Name: Post Login User 35
Scoring **37**
Service Name: Score Submission Service 37
Service Name: Get Team Submission History Service 40
Service Name: Get Team Level Submission History Service 40
Outputs: 41

Service Name: Get Submission Detail Service.....41
Service Name: Get Submission Log Service.....44
Service Name: Admin Get Submission Detail Service.....44
Service Name: Admin Get Submission Log Service.....46
Service Name: Admin Rescore Hackathon Service.....47
Service Name: Admin Get Recent Submissions Service.....48
Service Name: Get Level Leaderboard Service.....49
Service Name: Get Event Leaderboard Service.....50
Service Name: Event Leaderboard Update Stream Service.....51
Outputs:.....51
Announcements.....68
Dashboard.....72
Events.....75
Event Register.....81
Forums.....84
Superadmin.....94
Certificate.....96

Team Creation and Management

Service Name: Create Team Service

Description: Creates a new team for a specific event and automatically adds the creator as an approved member.

Inputs:

- teamName (string) - The name of the team (must be unique within the event).
- eventId (UUID) - The identifier of the event the team belongs to.
- currentUserId (UUID) - The identifier of the authenticated user creating the team.

Outputs:

- teamId (UUID) - Unique identifier of the created team.
- teamName (string) - Name of the team.
- eventId (UUID) - Event the team belongs to.
- createdByUserId (UUID) - ID of the user who created the team.
- createdAt (datetime) - Timestamp of creation.
- status (string) - Team status (e.g., "ACTIVE").
- joinCode (string) - Unique code that can be used by participants to request joining the team.

Usage / Interaction Rules:

- Clients must send a POST request to /api/teams with a JSON body containing teamName and eventId.
- The request must include authentication (HTTP Basic Auth or JWT). The user ID is extracted from the security context.
- Preconditions:
    - Team name must not already exist for the same event.
    - The user must not already be an approved member of another team in the same event.

- On success, returns HTTP 201 Created with the team object, including the generated joinCode.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Get My Teams Service

Description: Retrieves all teams the authenticated user belongs to, across all events.

Inputs:

- currentUserId (UUID) - Identifier of the authenticated user (extracted from the security context).

Outputs:

- A list of Team objects (same structure as Create Team Service output).

Usage / Interaction Rules:

- Clients must send a GET request to /api/teams/my-teams.
- The request must include authentication.
- On success, returns HTTP 200 OK with the JSON array of teams.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Get My Team For Event Service

Description: Retrieves the authenticated user's approved team for a specific event, if one exists.

Inputs:

- eventId (UUID) - Identifier of the event (query parameter).

currentUserId (UUID) - Identifier of the authenticated user (extracted from the security context).

Outputs:

- The Team object (same structure as Create Team Service output), or no content if the user has no team for the event.

Usage / Interaction Rules:

- Clients must send a GET request to /api/teams/my-team?eventId={eventId}.
- The request must include authentication.
- On success, returns HTTP 200 OK with the team, or HTTP 204 No Content if the user has no team for the event.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Request to Join Team Service

Description: Allows a participant to request joining an existing team. The request is stored as a pending membership.

Inputs:

- teamId (UUID) - Identifier of the team to join (in the URL path).
- currentUserId (UUID) - Identifier of the authenticated user requesting to join.

Outputs:
None (only HTTP status).

Usage / Interaction Rules:

- Clients must send a POST request to /api/teams/{teamId}/join-requests.
- The request must include authentication.
- Preconditions:
    - The team must exist and be active.
    - The user must not already have a pending or approved membership for this team.
    - The team must not have reached its maximum size (currently hardcoded to 4).
- On success, returns HTTP 201 Created with no body.
- On failure, returns an appropriate HTTP error with a message.

Service Name: View Join Requests Service

Description: Allows the team creator to view all pending join requests for their team.

Inputs:

- teamId (UUID) - Identifier of the team (in the URL path).
- currentUserId (UUID) - Identifier of the authenticated user (must be the team creator).

Outputs:

- A list of member objects (same structure as View Team Members Service), scoped to users with pending requests.

Usage / Interaction Rules:

- Clients must send a GET request to /api/teams/{teamId}/join-requests.
- The request must include authentication.
- On success, returns HTTP 200 OK with the JSON array of pending requests.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Request To Join Team By Code Service

Description: Allows a participant to request joining an existing team using its join code, instead of its teamId.

Inputs:

- joinCode (string) - The team's join code (in the URL path).
- currentUserId (UUID) - Identifier of the authenticated user requesting to join.

Outputs:

- None (only HTTP status).

Usage / Interaction Rules:

- Clients must send a POST request to /api/teams/join/{joinCode}.
- The request must include authentication.
- On success, returns HTTP 201 Created with no body.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Approve or Reject Join Request Service

Description: Allows the team creator to approve or reject a pending join request from another user.

Inputs:

- teamId (UUID) - Identifier of the team (in the URL path).
- userId (UUID) - Identifier of the user whose request is being decided (in the URL path).
- approve (boolean) - true to approve, false to reject (in the request body).
- currentUserId (UUID) - Identifier of the authenticated user (must be the team creator).

Outputs:
None (only HTTP status).

Usage / Interaction Rules:

- Clients must send a PUT request to /api/teams/{teamId}/join-requests/{userId} with a JSON body {"approve": true|false}.
- The request must include authentication.
- Preconditions:
    - The authenticated user must be the team creator.
    - The pending request must exist and have status PENDING.
    - If approving, the user must not already be an approved member of another team in the same event, and the team must still have capacity.
- On success, returns HTTP 200 OK with no body.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Leave Team Service

Description: Allows a user to leave a team. If the user is an approved member, the membership status changes to "LEFT". If the user has only a pending request, the membership record is deleted. If the last approved member leaves, the team becomes inactive.

Inputs:

- teamId (UUID) - Identifier of the team to leave (in the URL path).
- currentUserId (UUID) - Identifier of the authenticated user leaving.

Outputs:
None (only HTTP status).

Usage / Interaction Rules:

- Clients must send a DELETE request to /api/teams/{teamId}/members.
- The request must include authentication.
- Preconditions:
    - The user must be a member of the team (either pending or approved).
- On success, returns HTTP 204 No Content with no body.
- On failure, returns an appropriate HTTP error with a message.

Service Name: View Team Members Service

Description: Retrieves the list of approved members of a team, including their names, emails, join dates, and team role (LEADER or MEMBER).

Inputs:

- teamId (UUID) - Identifier of the team (in the URL path).

Outputs:

- A list of member objects, each containing:
    - userId (UUID) - User identifier.
    - fullName (string) - Concatenated first and last name.
    - email (string) - User's email address.
    - joinedAt (datetime) - When the user joined.
    - role (string) - Either "LEADER" or "MEMBER".

Usage / Interaction Rules:

- Clients must send a GET request to /api/teams/{teamId}/members.
- Authentication may be required (depending on security configuration).
- Preconditions:
    - The team must exist.
- On success, returns HTTP 200 OK with the JSON array of members.
- On failure (e.g., team not found), returns an appropriate HTTP error with a message.

File Storage - Specs and Submission Upload/Download

Service Name: Upload Level File Service

Description: Allows an Admin to upload an input or resource file for a specific event level. The file is stored in Azure Blob Storage and the storage key is returned for database persistence.

Inputs:

- hackathonId (UUID) - Identifier of the hackathon (in the URL path).
- levelId (short) - Identifier of the level (in the URL path). (previously documented as a generic string; it's the numeric levels.id)
- file (multipart/form-data) - The file to upload.
- fileType (string) - Type/category of the uploaded file.

Outputs:

- id (string) - New. The generated levelfiles.id.
- storageKey (string) - Blob path persisted to levelfiles.storage_key: hackathons/{hackathonId}/levels/{levelId}/{filename}.
- blobUrl (string) - The raw Azure blob URL.

Usage / Interaction Rules:

- Clients must send a POST request to /api/storage/hackathons/{hackathonId}/levels/{levelId}/files with a multipart/form-data body containing fields named file and fileType.
- The request must include authentication (JWT). Admin role required.
- On success, returns HTTP 200 OK with id, storageKey, and blobUrl.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Get Level File Download URL Service

Description: Returns a time-limited presigned SAS URL for a participant or admin to download a level input file directly from Azure Blob Storage.

Inputs:

- hackathonId (UUID) - Identifier of the hackathon (in the URL path).
- levelId (string) - Identifier of the level (in the URL path).
- filename (string) - The filename to retrieve (in the URL path).

Outputs:

- url (string) - A presigned SAS URL valid for 60 minutes.

Usage / Interaction Rules:

- Clients must send a GET request to /api/storage/hackathons/{hackathonId}/levels/{levelId}/files/{filename}.
- The request must include authentication (JWT). Admin or Participant role required.
- This endpoint does not query the database - it rebuilds the storage key deterministically from the path parameters. The filename supplied must exactly match the sanitised filename used at upload time.
- On success, returns HTTP 200 OK with the url field.
- On failure, returns an appropriate HTTP error with a message.

Service Name: List Level Files Service

Description: Lists all files uploaded for a specific level. Used by the admin Levels page to render each level's file list.

Inputs:

- hackathonId (UUID) - Identifier of the hackathon (in the URL path)
- levelId (long) - Identifier of the level (in the URL path)

Outputs:

- A list of LevelFile objects, each containing:
    - id (long) - Database identifier
    - levelId (long) - The level this file belongs to
    - fileName (string) - Original filename
    - storageKey (string) - Blob storage path
    - fileType (string) - File type (JSON)
    - fileSize (long) - Size in bytes
    - contentType (string) - MIME type
    - uploadedAt (datetime) - Upload timestamp

Usage / Interaction Rules:

- Clients must send a GET request to /api/storage/hackathons/{hackathonId}/levels/{levelId}/files
- The request must include authentication (JWT)
- Admin or Participant role required
- On success, returns HTTP 200 OK with the JSON array of level files
- On failure, returns an appropriate HTTP error with a message

Service Name: Delete Level File Service

Description: Deletes a level file, removing both the blob and its metadata record.

Inputs:

- hackathonId (UUID) - Identifier of the hackathon (in the URL path)
- levelId (long) - Identifier of the level (in the URL path)
- fileId (long) - Identifier of the level file to delete (in the URL path)

Outputs:

- None (only HTTP status)

Usage / Interaction Rules:

- Clients must send a DELETE request to /api/storage/hackathons/{hackathonId}/levels/{levelId}/files/{fileId}
- The request must include authentication (JWT)
- Admin role required
- On success, returns HTTP 204 No Content with no body
- On failure, returns an appropriate HTTP error with a message

Service Name: Upload Solver File Service

Description: Allows an Admin to upload a solver file for a specific hackathon. Supports versioning so hotfix solver versions can be uploaded without overwriting previous ones. Automatically deactivates all previous solver versions for the hackathon before saving the new one as active - only one solver version can be active per hackathon at a time.

Inputs:

- hackathonId (UUID) - Identifier of the hackathon (in the URL path)
- file (multipart/form-data) - The solver file to upload
- uploadedBy (UUID) - The admin performing the upload (extracted from authentication context)
- notes (string) - Optional release notes for this solver version

Outputs:

- solverVersionId (string) - The generated solverversion.id
- storageKey (string) - The blob path persisted to solverversion.storage_key: hackathons/{hackathonId}/solver/v{version}/{filename}
- blobUrl (string) - The raw Azure blob URL
- version (string) - The version number that was uploaded

Usage / Interaction Rules:

- Clients must send a POST request to /api/storage/hackathons/{hackathonId}/solver with a multipart/form-data body containing file, plus notes as optional request parameter
- The request must include authentication (JWT)
- Admin role required
- On success, returns HTTP 200 OK with solverVersionId, storageKey, blobUrl, and version

- On failure, returns an appropriate HTTP error with a message

Service Name: Upload Event Banner Service

Description: Allows an Admin to upload or replace the banner image for a specific event. Replaces the previous "Upload Branding Asset Service".

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).
- file (multipart/form-data) - The banner image to upload.

Outputs:

- storageKey (string) - The blob path of the uploaded banner.
- blobUrl (string) - The raw Azure blob URL.

Usage / Interaction Rules:

- Clients must send a POST request to /api/storage/events/{eventId}/banner with a multipart/form-data body containing a field named file.
- The request must include authentication (JWT). Admin role required.
- On success, returns HTTP 200 OK with storageKey and blobUrl.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Upload Event Logo Service

Description: Allows an Admin to upload or replace the brand logo image for a specific event.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).
- file (multipart/form-data) - The logo image to upload.

Outputs:

- storageKey (string) - The blob path of the uploaded logo.
- blobUrl (string) - The raw Azure blob URL.

Usage / Interaction Rules:

- Clients must send a POST request to /api/storage/events/{eventId}/logo with a multipart/form-data body containing a field named file.
- The request must include authentication (JWT). Admin role required.
- On success, returns HTTP 200 OK with storageKey and blobUrl.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Get Event Banner Download URL Service

Description: Returns a presigned SAS URL for an event's banner image, for display on the frontend.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).

Outputs:

- url (string) - A presigned SAS URL.
- storageKey (string) - The blob storage key of the banner.

Usage / Interaction Rules:

- Clients must send a GET request to /api/storage/events/{eventId}/banner.
- Authentication is not required.
- On success, returns HTTP 200 OK with the url and storageKey fields, or HTTP 204 No Content if no banner has been uploaded.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Get Event Logo Download URL Service

Description: Returns a presigned SAS URL for an event's brand logo image, for display on the frontend.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).

Outputs:

- url (string) - A presigned SAS URL.
- storageKey (string) - The blob storage key of the logo.

Usage / Interaction Rules:

- Clients must send a GET request to /api/storage/events/{eventId}/logo.
- Authentication is not required.
- On success, returns HTTP 200 OK with the url and storageKey fields, or HTTP 204 No Content if no logo has been uploaded.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Delete Event Banner Service

Description: Deletes an event's banner image, removing both the blob and clearing the stored key.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).

Outputs:

- None (only HTTP status).

Usage / Interaction Rules:

- Clients must send a DELETE request to /api/storage/events/{eventId}/banner.
- The request must include authentication (JWT). Admin role required.
- On success, returns HTTP 204 No Content.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Delete Event Logo Service

Description: Deletes an event's brand logo image, removing both the blob and clearing the stored key.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).

Outputs:

- None (only HTTP status).

Usage / Interaction Rules:

- Clients must send a DELETE request to /api/storage/events/{eventId}/logo.
- The request must include authentication (JWT). Admin role required.
- On success, returns HTTP 204 No Content.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Upload Problem Statement Service

Description: Uploads the problem statement PDF for a hackathon. The returned storageKey is saved to hackathon.problem_statement_storage_key, replacing any previous problem statement for this hackathon.

Inputs:

- hackathonId (UUID) - Identifier of the hackathon (in the URL path)
- file (multipart/form-data) - The uploaded PDF file (must be application/pdf)

Outputs:

- storageKey (string) - The blob path of the uploaded problem statement
- blobUrl (string) - The raw Azure blob URL

Usage / Interaction Rules:

- Clients must send a POST request to /api/storage/hackathons/{hackathonId}/problem-statement with a multipart/form-data body containing a field named file
- The request must include authentication (JWT)
- Admin role required

- The file must be a PDF (content-type: application/pdf)
- On success, returns HTTP 200 OK with storageKey and blobUrl
- On failure (e.g., not a PDF, no file provided), returns an appropriate HTTP error with a message

Service Name: Get Problem Statement Download URL Service

Description: Returns a presigned SAS URL for downloading a hackathon's problem statement PDF.

Inputs:

- hackathonId (UUID) - Identifier of the hackathon (in the URL path)

Outputs:

- url (string) - A presigned SAS URL valid for 60 minutes
- storageKey (string) - The blob storage key of the problem statement

Usage / Interaction Rules:

- Clients must send a GET request to /api/storage/hackathons/{hackathonId}/problem-statement
- The request must include authentication (JWT)
- Admin or Participant role required
- On success, returns HTTP 200 OK with the url and storageKey fields
- On failure (e.g., no problem statement uploaded), returns HTTP 404 Not Found

Service Name: Upload Submission Service

Description: Allows a Participant to submit a solution for a level in a single request - uploads both the solution output file and the zipped source code archive together, creates the submissions database record, and returns the generated submissionId.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path)
- teamId (UUID) - Identifier of the team (in the URL path)
- outputFile (multipart/form-data) - The solution output file (the artifact the solver grades)
- sourceFile (multipart/form-data) - A .zip archive of the source code
- levelId (short) - The level this submission is for

Outputs:

- submissionId (string) - The generated submissions.id
- outputStorageKey (string) - The blob path persisted to submissions.output_storage_key: submissions/{eventId}/{teamId}/levels/{levelId}/{submissionId}/output/{filename}
- sourceStorageKey (string) - The blob path persisted to submissions.source_code_storage_key, same shape with /source/ instead of /output/
- status (string) - The submission's status immediately after creation (currently "QUEUED")
- scoringRecordId (string) - The queue record identifier for the scoring job

Usage / Interaction Rules:

- Clients must send a POST request to /api/storage/events/{eventId}/teams/{teamId}/submissions with a multipart/form-data body containing outputFile and sourceFile, plus levelId as a request parameter
- The request must include authentication (JWT)
- Participant role required
- The platform never executes participant code. Only the output file is passed to the solver
- On success, returns HTTP 200 OK with submissionId, outputStorageKey, sourceStorageKey, status, and scoringRecordId
- On failure, returns an appropriate HTTP error with a message

Service Name: Get Submission Output Download URL Service

Description: Returns a presigned SAS URL for downloading a submission output file from Azure Blob Storage.

Inputs**:**

- eventId (UUID) - Identifier of the event (in the URL path)
- teamId (UUID) - Identifier of the team (in the URL path)
- submissionId (long) - Identifier of the submission (in the URL path)
- filename (string) - The output filename (in the URL path)

Outputs:

- url (string) - A presigned SAS URL valid for 60 minutes

Usage / Interaction Rules:

- Clients must send a GET request to /api/storage/events/{eventId}/teams/{teamId}/submissions/{submissionId}/output/{filename}
- The request must include authentication (JWT)
- Admin or Participant role required
- On success, returns HTTP 200 OK with the url field
- On failure, returns an appropriate HTTP error with a message

Service Name: Get Source Archive Download URL Service

Description: Returns a presigned SAS URL for downloading a source code archive from Azure Blob Storage. Admin only - used for post-event auditing to verify that the submitted output is reproducible by the team's code.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path)
- teamId (UUID) - Identifier of the team (in the URL path)
- submissionId (long) - Identifier of the submission (in the URL path)
- filename (string) - The archive filename (in the URL path)

Outputs:

- url (string) - A presigned SAS URL valid for 60 minutes

Usage / Interaction Rules:

- Clients must send a GET request to /api/storage/events/{eventId}/teams/{teamId}/submissions/{submissionId}/source/{filename}
- The request must include authentication (JWT)
- Admin role required
- On success, returns HTTP 200 OK with the url field
- On failure, returns an appropriate HTTP error with a message

Service Name: Get Scoring Log Download URL Service (Single Submission)

Description: Returns a presigned SAS URL for downloading a single submission's scoring log from Azure Blob Storage.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path)
- teamId (UUID) - Identifier of the team (in the URL path)
- levelId (string) - Identifier of the level (in the URL path)
- submissionId (string) - Identifier of the submission (in the URL path)

Outputs:

- url (string) - A presigned SAS URL valid for 60 minutes

Usage / Interaction Rules:

- Clients must send a GET request to /api/storage/events/{eventId}/teams/{teamId}/levels/{levelId}/submissions/{submissionId}
- The request must include authentication (JWT)
- Admin or Participant role required
- On success, returns HTTP 200 OK with the url field
- On failure, returns an appropriate HTTP error with a message

Admin Event Management

Service Name: Create Event Service

Description: Creates a new hackathon event with specific constraints

Inputs:

EventRequest object payload in the request body that contains:

- name (string) - The title of the event.
- registrationKey (string) - The optional passkey for private events.
- teamSizeLimit (short) - The maximum number of participants per team.
- startDateTime (OffsetDateTime) - The starting time and date of the event.
- duration (int) - The total duration of the event, in seconds.
- description (string) - Information about the specific hackathon event.
- visibility (string) - The visibility of the hackathon event to the participants.
- status (string) - The lifecycle of an event, it can be ACTIVE or INACTIVE as an example.
- inPerson (boolean) - Whether the event is in-person.
- allowedTech (list<string>) - Technologies allowed for submissions.
- rules (string) - Event rules text (also accepted as eventRules in the JSON body).
- tagline (string) - Short event tagline.
- firstPlacePrize (decimal) - First place prize amount.
- secondPlacePrize (decimal) - Second place prize amount.
- thirdPlacePrize (decimal) - Third place prize amount.
- totalPrizePool (decimal) - Total prize pool amount.
- freezeTime (datetime) - The time at which the leaderboard should freeze.

- hackathonId (UUID) - Identifier of the parent hackathon this event belongs to.

Output:

- eventId (UUID) - Automatically generated unique database identifier for the event.
- createdByUserId (UUID) - The identifier of the admin that created the event.
- name (string) - The configured title of the event.
- registrationKey (string) - The password for the event, or null if visibility is not private.
- teamSizeLimit (short) - The configured team size constraint for the event.
- startDateTime (OffsetDateTime) - The configured start date for the event.
- duration (int) - The configured duration of the event, in seconds.
- endDateTime (OffsetDateTime) - Computed event end date/time (startDateTime + duration).
- description (string) - The configured description of the event.
- visibility (string) - The configured visibility of the event.
- status (string) - The configured status of the event.
- inPerson (boolean) - The configured in-person flag.
- allowedTech (list<string>) - The configured list of allowed technologies.
- rules (string) - The configured rules text.
- tagline (string) - The configured tagline.
- firstPlacePrize (decimal) - The configured first place prize.
- secondPlacePrize (decimal) - The configured second place prize.
- thirdPlacePrize (decimal) - The configured third place prize.
- totalPrizePool (decimal) - The configured total prize pool.
- freezeTime (datetime) - The configured leaderboard freeze time.
- bannerStorageKey (string) - Storage key for the event banner image, if set.
- logoStorageKey (string) - Storage key for the event logo image, if set.

Usage / Interaction Rules:

- Clients must send a POST request to /api/admin/events.

- The body of the request must include a valid JSON object matching the properties expected by EventRequest.
- The client must hold a valid admin role (ROLE_ADMIN).
- On success: Returns HTTP 201 Created with the newly created event in the response body.
- On failure: Returns appropriate HTTP error payload.

Service Name: Get Admin Events Service

Description: Retrieves a list of all hackathon events that were created by a specific admin, this is done by using their specific UUID and matching it to the createdByUserId field.

Inputs:

- createdByUserId (UUID) - the unique identifier of the admin whose events are being retrieved.

Output:

- Returns a List<Event> array in the response body. Each event in the body is a valid event that satisfies the rules in the Create Event Service. If the admin has not yet created any events an empty array will be returned.

Usage / Interaction Rules:

- Client must send a GET request to the /api/admin/events.
- The client must hold a valid admin role (ROLE_ADMIN).
- On success: Returns HTTP 200 OK containing the JSON array of the matching event entities (or an empty array if no events could be found for an admin).
- On failure: Returns HTTP 403 Forbidden if the client does not have a ROLE_ADMIN.
- On failure: HTTP 400 if the id provided is malformed or not a valid UUID.

Service Name: Update Event Service

Description: Performs a complete overwrite of an existing hackathon event's configured data using the provided event identifier and request details.

Input:

- EventRequest object payload in the request body (same fields as Create Event Service).

Output:

- Returns the updated and fully persistent Event entity object with the updated information matching that specified eventId (same structure as Create Event Service output).

Usage / Interaction Rules:

- Clients must send a PUT request to /api/admin/events/{id} where id is the UUID of the specific event that exists on the database.
- The event must already be present in the database.
- On success: Return HTTP 200 OK containing the updated Event entity representation inside the response body payload.
- On failure: Returns HTTP 403 Forbidden if the client does not have a ROLE_ADMIN.
- On failure: HTTP 404 Not Found if the passed in eventId is not in the database or could not be found.

Service Name: Patch Event Status Service

Description: Performs a partial update on an existing hackathon event to modify its status, visibility or registrationKey if needed. The rest of the event stays unchanged.

Input:

An EventRequest object payload in the request body that contains:

- visibility (string) - The new visibility state.
- status (string) - the new status state.
- registrationKey (string) - the updated passkey to the event (this is optional and only required if the event visibility is private)
- Note: only these three fields are read by this endpoint, even though the request body is a full EventRequest object - the rest of the event's fields are left unchanged.

Output:

- Returns the modified and fully persistent Event entity object with the updated information matching that specified eventId (same structure as Create Event Service output).

Usage / Interaction Rules:

- Client must send a PATCH request to /api/admin/events/{id}/status where id represents the target events id.
- If the visibility is private then a registrationKey must be provided, otherwise it will automatically be set to null.
- On success: Return HTTP 200 OK containing the modified Event entity representation inside the response body payload.
- On failure: Returns 404 Not Found if the eventId does not exist in the database.
- On failure: On failure: Returns HTTP 403 Forbidden if the client does not have a ROLE_ADMIN.

Service Name: Get Event Status Service

Description: Retrieves a lightweight summary containing only the current operational status and event visibility of a specific hackathon event.

Input:

- eventId (UUID) - the unique identifier of the specific event we are inspecting.

Output:

- Status (string) - the current status of the event we are inspecting.
- Visibility (String) - the current visibility state of the event we are inspecting.

Usage / Interaction Rules:

- Clients must send a GET request to /api/admin/events/{id}/status where id is the UUID of the specified event.
- The endpoint is restricted to authenticated users with the role admin (ROLE_ADMIN).
- On success: HTTP 200 OK containing the EventStatusResponse JSON payload
- On failure: Returns 404 Not Found if the eventId does not exist in the database.
- On failure: On failure: Returns HTTP 403 Forbidden if the client does not have a ROLE_ADMIN.

Authorization

Service Name: Post Register User

Description: Registers a user by entering their credentials into the system. Assigns a Participant role by default.

Input:

- firstName (string) - First name of the user.
- lastName (String) - Last name of the user.
- Email (String) - Email of the user.
- Password (String) - Password of the user.

Output:

- Token (string) - JWT token.
- UserId (UUID) - IDof the user.
- firstName (string) - First name of the user.
- lastName (String) - Last name of the user.
- Email (String) - Email of the user.
- Role (string) - The user's role.

Usage / Interaction Rules:

- Clients must send a POST request to /api/auth/register.
- The endpoint assigns a JWT token.
- On success: HTTP 200 OK containing the AuthResponse JSON payload
- On failure: Returns 409 Conflict if the user is already registered.

Service Name: Post Login User

Description: Login using saved email and password.

Input:

- Email (string) - Users email.
- Password (string) - Users password.

Output:

- Token (string) - JWT token.
- UserId (UUID) - IDof the user.
- firstName (string) - First name of the user.
- lastName (String) - Last name of the user.
- Email (String) - Email of the user.
- Role (string) - The user's role.

Usage / Interaction Rules:

- Clients must send a POST request to /api/auth/login.
- The endpoint assigns a JWT token.
- On success: HTTP 200 OK containing the AuthResponse JSON payload
- On failure: Returns 404 Not Found if the user is not registered.

Service Name: Get Current User Service

Description: Retrieves the profile of the currently authenticated user.

Inputs:

- currentUserId (UUID) - Identifier of the authenticated user (extracted from the security context).

Outputs:

- token (string) - JWT token.
- userId (UUID) - ID of the user.
- firstName (string) - First name of the user.
- lastName (string) - Last name of the user.
- email (string) - Email of the user.
- role (string) - The user's role.

Usage / Interaction Rules:

- Clients must send a GET request to /api/auth/me.
- The request must include authentication (JWT).
- On success, returns HTTP 200 OK with the AuthResponse JSON payload.
- On failure, returns an appropriate HTTP error with a message.

Scoring

Service Name: Score Submission Service

Description: Triggers scoring for a submission: runs the active solver against the submission's output file and persists score, status and logs. Safe to call again later (e.g. admin-triggered re-scoring after a solver hotfix) - it only overwrites this submission's own log, no other submission's log is touched.

Inputs:

- submissionId (long) - Identifier of the submission to score (in the URL path)

Outputs:

- submissionId (string) - The identifier of the submission being scored
- status (string) - The submission status ("QUEUED")
- recordId (string) - The queue record identifier for the scoring job

Usage / Interaction Rules:

- Clients must send a POST request to /api/scoring/submissions/{submissionId}/score
- The request must include authentication (JWT)
- Admin or Participant role required
- On success, returns HTTP 202 Accepted with submissionId, status, and recordId
- On failure, returns an appropriate HTTP error with a message

Service Name: Pause Scoring Service

Description: Allows an Admin to pause scoring for an event, so newly submitted work is queued but not scored until resumed.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).

Outputs:

- eventId (UUID) - Identifier of the event.
- scoringPaused (boolean) - Whether scoring is now paused (true).

Usage / Interaction Rules:

- Clients must send a PATCH request to /api/admin/events/{id}/scoring/pause.
- The request must include authentication (JWT). Admin role required.
- On success, returns HTTP 200 OK with the ScoringPauseResponse payload.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Resume Scoring Service

Description: Allows an Admin to resume scoring for an event that was previously paused.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).

Outputs:

- eventId (UUID) - Identifier of the event.
- scoringPaused (boolean) - Whether scoring is now paused (false).

Usage / Interaction Rules:

- Clients must send a PATCH request to /api/admin/events/{id}/scoring/resume.
- The request must include authentication (JWT). Admin role required.
- On success, returns HTTP 200 OK with the ScoringPauseResponse payload.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Get Team Submission History Service

Description: Full submission history for a team across all levels, most recent first. Used by the participant portal's "submission history" view.

Inputs:

- teamId (UUID) - Identifier of the team (in the URL path)

Outputs:

- A list of SubmissionResponse objects, each containing:
  - submissionId (long) - Submission identifier
  - teamId (UUID) - Team identifier
  - levelId (short) - Level identifier
  - solverVersionId (long) - Solver version used
  - score (BigDecimal) - Score achieved
  - status (string) - Submission status (QUEUED, PROCESSING, SCORED, FAILED)
  - submittedAt (datetime) - Submission timestamp
  - outputFileName (string) - Output filename
  - sourceFileName (string) - Source archive filename
  - scoringLog (ScoringLogResponse) - Scoring log (null in history view)

Usage / Interaction Rules:

- Clients must send a GET request to /api/scoring/teams/{teamId}/submissions
- The request must include authentication (JWT)
- Admin or Participant role required
- On success, returns HTTP 200 OK with the JSON array of submissions
- On failure, returns an appropriate HTTP error with a message

Service Name: Get Team Level Submission History Service

Description: Submission history for a team, scoped to a single level, most recent first.

Inputs:

- teamId (UUID) - Identifier of the team (in the URL path)
- levelId (short) - Identifier of the level (in the URL path)

Outputs:

- A list of SubmissionResponse objects (same structure as Get Team Submission History Service)

Usage / Interaction Rules:

- Clients must send a GET request to /api/scoring/teams/{teamId}/levels/{levelId}/submissions
- The request must include authentication (JWT)
- Admin or Participant role required
- On success, returns HTTP 200 OK with the JSON array of submissions
- On failure, returns an appropriate HTTP error with a message

Service Name: Get Submission Detail Service

Description: Full feedback for a single submission, including score, status and its scoring log. Scoped to the owning team so a participant can't view another team's feedback by guessing IDs.

Inputs:

- teamId (UUID) - Identifier of the team (in the URL path)
- submissionId (long) - Identifier of the submission (in the URL path)

Outputs:

- A SubmissionResponse object containing:
    - submissionId (long) - Submission identifier
    - teamId (UUID) - Team identifier
    - levelId (short) - Level identifier
    - solverVersionId (long) - Solver version used
    - score (BigDecimal) - Score achieved
    - status (string) - Submission status (QUEUED, PROCESSING, SCORED, FAILED)
    - submittedAt (datetime) - Submission timestamp
    - outputFileName (string) - Output filename
    - sourceFileName (string) - Source archive filename
    - scoringLog (ScoringLogResponse) - Scoring log with content

Usage / Interaction Rules:

- Clients must send a GET request to /api/scoring/teams/{teamId}/submissions/{submissionId}
- The request must include authentication (JWT)
- Admin or Participant role required
- On success, returns HTTP 200 OK with the SubmissionResponse JSON payload
- On failure, returns an appropriate HTTP error with a message

Service Name: Get Recent Submissions For Event Service

Description: Retrieves recent submissions for a single event for admin monitoring, with the team name and level number/name already resolved for display.

Inputs:

- eventId (UUID) - Identifier of the event to scope submissions to (in the URL path).
- limit (int) - Maximum number of submissions to retrieve (in the URL path).

Outputs:

- A list of RecentSubmissionResponse objects, each containing:
- submissionId (long) - Submission identifier.
- teamId (UUID) - Team identifier.
- teamName (string) - Name of the team.
- eventId (UUID) - Event identifier.
- eventName (string) - Name of the event.
- levelId (short) - Level identifier.
- levelNumber (short) - Level number/order.
- levelName (string) - Name of the level.
- score (BigDecimal) - Score achieved.
- status (string) - Submission status.
- submittedAt (datetime) - Submission timestamp.

Usage / Interaction Rules:

- Clients must send a GET request to /api/scoring/admin/events/{eventId}/recentsubmissions/{limit}.
- The request must include authentication (JWT). Admin role required.
- On success, returns HTTP 200 OK with the JSON array of recent submissions.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Get Submission Log Service

Description: Just the scoring log for a single submission (score/status not included). Scoped to the owning team so a participant can't view another team's log by guessing IDs.

Inputs:

- teamId (UUID) - Identifier of the team (in the URL path)
- submissionId (long) - Identifier of the submission (in the URL path)

Outputs:

- A ScoringLogResponse object containing:
  - submissionId (long) - Submission identifier
  - teamId (UUID) - Team identifier
  - eventId (UUID) - Event identifier
  - logPath (string) - Blob storage path of the log
  - scoredAt (datetime) - Scoring timestamp
  - logContent (string) - The content of the scoring log

Usage / Interaction Rules:

- Clients must send a GET request to /api/scoring/teams/{teamId}/submissions/{submissionId}/log
- The request must include authentication (JWT)
- Admin or Participant role required
- On success, returns HTTP 200 OK with the ScoringLogResponse JSON payload
- On failure (e.g., log not found), returns HTTP 404 Not Found

Service Name: Admin Get Submission Detail Service

Description: Admin variant: full feedback for any submission regardless of team, for support/auditing.

Inputs:

- submissionId (long) - Identifier of the submission (in the URL path)

Outputs:

- A SubmissionResponse object (same structure as Get Submission Detail Service)

Usage / Interaction Rules:

- Clients must send a GET request to /api/scoring/admin/submissions/{submissionId}
- The request must include authentication (JWT)
- Admin role required
- On success, returns HTTP 200 OK with the SubmissionResponse JSON payload
- On failure, returns an appropriate HTTP error with a message

Service Name: Admin Get Submission Log Service

Description: Admin variant: just the scoring log for any submission regardless of team.

Inputs:

- submissionId (long) - Identifier of the submission (in the URL path)

Outputs:

- A ScoringLogResponse object (same structure as Get Submission Log Service)

Usage / Interaction Rules:

- Clients must send a GET request to /api/scoring/admin/submissions/{submissionId}/log
- The request must include authentication (JWT)
- Admin role required
- On success, returns HTTP 200 OK with the ScoringLogResponse JSON payload
- On failure (e.g., log not found), returns HTTP 404 Not Found

Service Name: Admin Rescore Hackathon Service

Description: Re-enqueues every submission for all events under a hackathon (e.g. after a solver hotfix).

Inputs:

- hackathonId (UUID) - Identifier of the hackathon whose submissions should be rescored (in the URL path)

Outputs:

- hackathonId (string) - Identifier of the hackathon being rescored
- queuedCount (int) - Number of submissions that were enqueued
- status (string) - The operation status ("QUEUED")

Usage / Interaction Rules:

- Clients must send a POST request to /api/scoring/admin/hackathons/{hackathonId}/rescore
- The request must include authentication (JWT)
- Admin role required
- On success, returns HTTP 202 Accepted with hackathonId, queuedCount, and status
- On failure, returns an appropriate HTTP error with a message

Service Name: Admin Get Recent Submissions Service

Description: Retrieves recent submissions for admin monitoring.

Inputs:

- limit (int) - Maximum number of submissions to retrieve (in the URL path)

Outputs:

- A list of SubmissionResponse objects (same structure as Get Team Submission History Service, without scoring logs)

Usage / Interaction Rules:

- Clients must send a GET request to /api/scoring/admin/recentsubmissions/{limit}
- The request must include authentication (JWT)
- Admin role required
- On success, returns HTTP 200 OK with the JSON array of recent submissions
- On failure, returns an appropriate HTTP error with a message

Service Name: Get Level Leaderboard Service

Description: Retrieves the leaderboard for a specific level within an event.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path)
- levelId (short) - Identifier of the level (in the URL path)

Outputs:

- A list of LeaderboardEntryResponse objects, each containing:
    - rank (int) - Ranking position
    - teamId (UUID) - Team identifier
    - teamName (string) - Name of the team
    - bestScore (BigDecimal) - Best score achieved
    - lastScoredAt (datetime) - Timestamp of the last scoring

Usage / Interaction Rules:

- Clients must send a GET request to /api/scoring/events/{eventId}/levels/{levelId}/leaderboard
- The request must include authentication (JWT)
- Admin or Participant role required
- On success, returns HTTP 200 OK with the JSON array of leaderboard entries
- On failure, returns an appropriate HTTP error with a message

Service Name: Get Event Leaderboard Service

Description: Retrieves the overall leaderboard for an event (all levels combined).

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path)

Outputs:

- A list of LeaderboardEntryResponse objects (same structure as Get Level Leaderboard Service)

Usage / Interaction Rules:

- Clients must send a GET request to /api/scoring/events/{eventId}/leaderboard
- The request must include authentication (JWT)
- Admin or Participant role required
- On success, returns HTTP 200 OK with the JSON array of leaderboard entries
- On failure, returns an appropriate HTTP error with a message

Service Name: Event Leaderboard Update Stream Service

Description: Server-Sent Events stream for real-time leaderboard updates for an event.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path)

Outputs:

- Server-Sent Events (SSE) stream of leaderboard updates. Each event contains updated leaderboard data.

Usage / Interaction Rules:

- Clients must send a GET request to /api/scoring/events/{eventId}/leaderboard/update
- The request must include authentication (JWT)
- Admin or Participant role required
- The endpoint produces MediaType.TEXT_EVENT_STREAM_VALUE
- On success, returns an SSE stream with leaderboard updates
- On failure, returns an appropriate HTTP error with a message

Service Name: Get Level Leaderboard Service

Description:

- Retrieves the leaderboard for a specific level within an event. Ranks the teams according to their best scored submission for that level. The highest score will get the best rank (i.e. 1).

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path)
- levelId (Long) - Identifier of the level (in the URL path)

Outputs:

- A list of LeaderboardEntryResponse objects, each containing:
    - Rank (int) - the team position on the leaderboard.
    - teamId (UUID) - identifier of the team.
    - teamName (string) - the name of the team.
    - bestScore (BigDecimal) - the best score achieved by the team this event
    - lastScoredAt (datetime) - Timestamp of when submission was scored, may be null.

Usage / Interaction Rules:

- Client must send a GET request to: /api/scoring/events/{eventId}/levels/{levelId}/leaderboard
- The request must include authentication (JWT)
- Admin or Participant role required
- Only submissions with status SCORED are included in the leaderboard calculation.
- If a team has not been scored yet, they automatically get a score of 0 on the leaderboard.
- Results are ordered by bestScore in descending order.

- On success, return HTTP 200 OK with a JSON array of leaderboard entries.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Get Event Leaderboard Service

Description:

- Retrieves the leaderboard for a specific event across all levels. Ranks the teams according to their best scored submission for that level. The highest score will get the best rank (i.e. 1).

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path)

Outputs:

- Rank (int) - the team position on the leaderboard.
- teamId (UUID) - identifier of the team.
- teamName (string) - the name of the team.
- bestScore (BigDecimal) - the best score achieved by the team this event
- lastScoredAt (datetime) - Timestamp of when submission was scored, may be null.

Usage / Interaction Rules:

- Client must send a GET request to: /api/scoring/events/{eventId}/leaderboard
- The request must include authentication (JWT)
- Admin or Participant role required
- Only submissions with status SCORED are included in the leaderboard calculation.
- The final best score is computed by summing the team's best score across all levels.
- If a team has not been scored yet, they automatically get a score of 0 on the leaderboard.
- Results are ordered by bestScore in descending order.

- On success, return HTTP 200 OK with a JSON array of leaderboard entries.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Get Level Leaderboard Service

Description:

- Provides SSE stream for real-time leaderboard update notifications. This allows the connected clients to be notified when the leaderboard might have changed after a submission has been graded.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path)

Outputs:

- SSE event stream
- Each leaderboard-update event contains update metadata:
    - eventId (UUID) - identifier of the event whose leaderboard has been updated.
    - levelId (Long) - identifier of the level affected by the scored submission.
    - teamId (UUID) - Identifier of the team whose submission was scored.
    - submissionId (Long) - Identifier of the scored submission.
    - OccurredAt (datetime) - Timestamp of when the update event was issued.

Usage / Interaction Rules:

- Client must send a GET request to: /api/scoring/events/{eventId}/leaderboard/update
- The request must include authentication (JWT)
- Admin or Participant role required
- The endpoint produces: MediaType.TEXT_EVENT_STREAM_VALUE
- The connection remains open as long as the client is subscribed to leaderboard updates

- When a submission is successfully scored, the backend sends a leaderboard-update event to connect clients for the relevant event
- SSE acts as a notification that the leaderboard has changed
- After receiving the event, the client should refresh the leaderboard by calling: GET /api/scoring/events/{eventId}/leaderboard
- On Success, returns SSE stream with leaderboard update notifications
- On Failure, returns an appropriate HTTP error with a message

Service Name: Create Hackathon Service

Description:

- Creates a new hackathon that participants can register for and administrators can manage.

Inputs:

- HackathonRequest - JSON request body containing the hackathon details.

Outputs:

- Hackathon - The newly created hackathon object.

Usage / Interaction Rules:

- Client must send a POST request to:
  /api/hackathon
- The request must include authentication (JWT).
- Admin role required.
- Request body must contain valid hackathon information.
- On success, returns HTTP 200 OK with the created hackathon.
- On failure, returns an appropriate HTTP error message.

Service Name: Get All Hackathons Service

Description:

- Retrieves all hackathons available in the system.

Inputs:

- None

Outputs:

- Hackathon[] - List of hackathons.

Usage / Interaction Rules:

- Client must send a GET request to:
  /api/hackathon
- Authentication is not required.
- Returns HTTP 200 OK with a JSON array of hackathons.
- On failure, returns an appropriate HTTP error message.

Service Name: Get Hackathon Service

Description:

- Retrieves a specific hackathon by its identifier.

Inputs:

- hackathonId (UUID) - Identifier of the hackathon (URL path).

Outputs:

- Hackathon - Requested hackathon.

Usage / Interaction Rules:

- Client must send a GET request to: /api/hackathon/{hackathonId}
- Authentication is not required.
- Returns HTTP 200 OK with the hackathon.
- Returns an appropriate HTTP error if the hackathon cannot be found.

Service Name: Update Hackathon Service

Description:

- Update Hackathon Service

Inputs:

- hackathonId (UUID) - Identifier of the hackathon.
- HackathonRequest- Updated hackathon information.

Outputs:

- Hackathon- Updated hackathon.

Usage / Interaction Rules:

- Client must send a PUT request to: /api/hackathon/{hackathonId}
- The request must include authentication (JWT).
- Admin role required.
- Returns HTTP 200 OK with the updated hackathon.
- On failure, returns an appropriate HTTP error message.

Service Name: Delete Hackathon

Description:

- Deletes an existing hackathon.

Inputs:

- hackathonId (UUID)- Identifier of the hackathon.

Outputs:

- None

Usage / Interaction Rules:

- Client must send a DELETE request to: /api/hackathon/{hackathonId}
- The request must include authentication (JWT).
- Admin role required.
- Returns HTTP 204 No Content on success.
- On failure, returns an appropriate HTTP error message.

Service Name: Create Event

Description:

- Creates a new event for a specified hackathon.

Inputs:

- hackathonId (UUID)- Identifier of the hackathon.
- EventRequest- Event details.

Outputs:

- Event - Newly created event.

Usage / Interaction Rules:

- Client must send a POST request to: /api/hackathon/{hackathonId}/events
- The request must include authentication (JWT).
- Admin role required.
- The supplied hackathon ID is associated with the event before creation.
- Returns HTTP 200 OK with the created event.
- On failure, returns an appropriate HTTP error message.

Service Name: Get Hackathon Events Service

Description:

- Retrieves all events belonging to a specific hackathon.

Inputs:

- hackathonId (UUID)- Identifier of the hackathon.

Outputs:

- Event[]- List of events.

Usage / Interaction Rules:

- Client must send a GET request to: /api/hackathon/{hackathonId}/events
- Authentication is not required.

- Returns HTTP 200 OK with a JSON array of events.
- On failure, returns an appropriate HTTP error message.

Service Name: Create Level

Description:

- Creates a new competition level for a hackathon.

Inputs:

- hackathonId (UUID)- Identifier of the hackathon.
- LevelRequest- Level details.

Outputs:

- Level- Newly created level.

Usage / Interaction Rules:

- Client must send a POST request to: /api/hackathons/{hackathonId}/levels
- The request must include authentication (JWT).
- Admin role required.
- Returns HTTP 200 OK with the created level.
- On failure, returns an appropriate HTTP error message.

Service Name: Update Level

Description:

- Updates an existing competition level.

Inputs:

- levelId (short)- Identifier of the level.
- LevelRequest- Updated level details.

Outputs:

- Level- Updated level.

Usage / Interaction Rules:

- Client must send a PUT request to: /api/levels/{levelId}
- The request must include authentication (JWT).
- Admin role required.
- Returns HTTP 200 OK with the updated level.
- On failure, returns an appropriate HTTP error message.

Service Name: Delete Level

Description:

- Deletes a competition level.

Inputs:

- levelId (short)- Identifier of the level.

Outputs:

- None

Usage / Interaction Rules:

- Client must send a DELETE request to: /api/levels/{levelId}
- The request must include authentication (JWT).
- Admin role required.
- Returns HTTP 204 No Content on success.
- On failure, returns an appropriate HTTP error message.

Service Name: Get Hackathon Levels

Description:

- Retrieves all competition levels associated with a hackathon.

Inputs:

- hackathonId (UUID)- Identifier of the hackathon.

Outputs:

- Level[]- List of competition levels.

Usage / Interaction Rules:

- Client must send a GET request to: /api/hackathons/{hackathonId}/levels
- Authentication is not required.
- Returns HTTP 200 OK with a JSON array of levels.
- On failure, returns an appropriate HTTP error message.

Service Name: Get Level

Description:

- Retrieves the details of a specific competition level.

Inputs:

- levelId (short)- Identifier of the level.

Outputs:

- Level- Requested competition level.

Usage / Interaction Rules:

- Client must send a GET request to:
  /api/levels/{levelId}
- Authentication is not required.
- Returns HTTP 200 OK with the requested level.
- Returns an appropriate HTTP error message if the level cannot be found.

Announcements

Service Name: Get Announcements Service

Description: Retrieves the list of announcements for an event that are visible to the requesting participant.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).
- currentUserId (UUID) - Identifier of the authenticated user (extracted from the security context).

Outputs:

- A list of Announcement objects, each containing:
- messageId (UUID) - Unique identifier of the announcement.
- eventId (UUID) - Event the announcement belongs to.
- title (string) - Announcement title.
- body (string) - Announcement content.
- severity (string) - One of "INFO", "IMPORTANT" or "URGENT".
- createdAt (datetime) - Timestamp of creation.

Usage / Interaction Rules:

- Clients must send a GET request to /api/events/{eventId}/announcements.
- The request must include authentication (JWT). Participant role required.
- On success, returns HTTP 200 OK with the JSON array of announcements.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Stream Announcements Service

Description: Server-Sent Events stream that pushes new announcements to a participant in real time for a given event.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).
- currentUserId (UUID) - Identifier of the authenticated user (extracted from the security context).

Outputs:

- Server-Sent Events (SSE) stream. Each event contains a newly published announcement.

Usage / Interaction Rules:

- Clients must send a GET request to /api/events/{eventId}/announcements/stream.
- The request must include authentication (JWT). Participant role required.
- The user must have access to the event, otherwise the connection is rejected.
- The endpoint produces MediaType.TEXT_EVENT_STREAM_VALUE.
- On success, returns an SSE stream with announcement events.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Create Announcement Service (Admin)

Description: Allows an Admin to publish a new announcement to all participants of an event and notifies subscribers of the announcement stream.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).
- title (string) - Announcement title (required, max 150 characters).
- body (string) - Announcement content (required).
- severity (string) - One of "INFO", "IMPORTANT" or "URGENT" (defaults to "INFO").
- currentUserId (UUID) - Identifier of the authenticated admin (extracted from the security context).

Outputs:

- announcement (Announcement) - The created announcement object (messageId, eventId, title, body, severity, createdAt).
- emailRecipientCount (int) - Number of participants emailed about the announcement.
- emailStatus (string) - Status of the email dispatch.

Usage / Interaction Rules:

- Clients must send a POST request to /api/admin/events/{eventId}/announcements with a JSON body containing title, body, and optionally severity.
- The request must include authentication (JWT). Admin role required.
- On success, returns HTTP 201 Created with the CreateAnnouncementResponse payload.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Get Announcements Service (Admin)

Description: Retrieves all announcements created for an event, for administrative review.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).
- currentUserId (UUID) - Identifier of the authenticated admin (extracted from the security context).

Outputs:

- A list of Announcement objects (same structure as Get Announcements Service).

Usage / Interaction Rules:

- Clients must send a GET request to /api/admin/events/{eventId}/announcements.
- The request must include authentication (JWT). Admin role required.
- On success, returns HTTP 200 OK with the JSON array of announcements.
- On failure, returns an appropriate HTTP error with a message.

Dashboard

Service Name: Get Admin Dashboard Service

Description: Retrieves a wide overview of platform activity across every event created by the admin.

Inputs:

- currentUserId (UUID) - Identifier of the authenticated admin (extracted from the security context).

Outputs:

- activeEvents (long) - Number of currently active events.
- totalEvents (long) - Total number of events created by the admin.
- totalParticipants (long) - Total number of participants across those events.
- submissionsToday (long) - Number of submissions made today.
- totalSubmissions (long) - Total number of submissions across those events.

Usage / Interaction Rules:

- Clients must send a GET request to /api/admin/dashboard.
- The request must include authentication (JWT). Admin role required.
- On success, returns HTTP 200 OK with the AdminDashboardResponse JSON payload.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Get Event Insights Service

Description: Retrieves detailed dashboard statistics for a single event, including submission trends and per-level score distribution.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).
- trendWindowMinutes (int) - Size, in minutes, of the trend window used to bucket the submission rate (query parameter, defaults to 60).

Outputs:

- eventId (UUID) - Identifier of the event.
- activeTeams (long) - Number of active teams in the event.
- approvedParticipants (long) - Number of approved participants in the event.
- totalSubmissions (long) - Total number of submissions for the event.
- submissionsLastHour (long) - nNumber of submissions made in the last hour.
- submissionsByStatus (map<string, long>) - Count of submissions grouped by status.
- errorRate (double) - Proportion of submissions that failed scoring.
- submissionRate (list) - Time-bucketed submission counts, each containing:
- bucketStart (datetime) - Start of the time bucket.
- count (long) - Number of submissions in that bucket.
- scoreDistributionByLevel (list) - Per-level score statistics, each containing:
- levelId (short) - Identifier of the level.
- levelName (string) - Name of the level.
- scoredSubmissions (long) - Number of scored submissions for the level.

- minScore (BigDecimal) - Minimum score achieved.
- maxScore (BigDecimal) - Maximum score achieved.
- avgScore (BigDecimal) - Average score achieved.

Usage / Interaction Rules:

- Clients must send a GET request to /api/admin/events/{id}/insights, with an optional trendWindowMinutes query parameter.
- The request must include authentication (JWT). Admin role required.
- On success, returns HTTP 200 OK with the EventInsightsResponse JSON payload.
- On failure, returns an appropriate HTTP error with a message.

Events

Service Name: Get Open Events Service

Description: Retrieves all events currently open and visible to participants.

Inputs: None.

Outputs:

A list of Event objects, each containing:

- eventId (UUID) - Unique identifier of the event.
- hackathon (UUID) - Identifier of the parent hackathon.
- createdByUserId (UUID) - Identifier of the admin who created the event.
- name (string) - Event name.
- teamSizeLimit (short) - Maximum number of members per team.
- startDateTime (datetime) - Event start date/time.
- duration (int) - Event duration, in seconds.
- endDateTime (datetime) - Computed event end date/time (startDateTime + duration).
- description (string) - Event description.
- visibility (string) - Event visibility (e.g. "PUBLIC", "PRIVATE").
- status (string) - Event status (e.g. "OPEN", "ACTIVE", "COMPLETED").
- scoringPaused (boolean) - Whether scoring is currently paused.
- inPerson (boolean) - Whether the event is in-person.
- allowedTech (list<string>) - Technologies allowed for submissions.
- rules (string) - Event rules text.
- tagline (string) - Short event tagline.
- firstPlacePrize (decimal) - First place prize amount.
- secondPlacePrize (decimal) - Second place prize amount.

- thirdPlacePrize (decimal) - Third place prize amount.
- totalPrizePool (decimal) - Total prize pool amount.
- leaderboardFreezeDateTime (datetime) - Time at which the leaderboard freezes, if set.
- bannerStorageKey (string) - Storage key for the event banner image, if set.
- logoStorageKey (string) - Storage key for the event logo image, if set.

Usage / Interaction Rules:

- Clients must send a GET request to /api/events/open.
- Authentication is not required.
- On success, returns HTTP 200 OK with the JSON array of open events.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Get User Active Events Service

Description: Retrieves the events the authenticated participant is currently registered for and that are still active.

Inputs:

- currentUserId (UUID) - Identifier of the authenticated user (extracted from the security context).

Outputs:

- A list of Event objects. (same as objects in Get Open Events Service service)

Usage / Interaction Rules:

- Clients must send a GET request to /api/events/user-active-events.
- The request must include authentication (JWT).
- On success, returns HTTP 200 OK with the JSON array of active events.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Get User Completed Events Service

Description: Retrieves the events the authenticated participant registered for that have since ended.

Inputs:

- currentUserId (UUID) - Identifier of the authenticated user (extracted from the security context).

Outputs:

- A list of Event objects. (same as objects in Get Open Events Service service)

Usage / Interaction Rules:

- Clients must send a GET request to /api/events/completed.
- The request must include authentication (JWT).
- On success, returns HTTP 200 OK with the JSON array of completed events.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Get Event By Id Service

Description: Retrieves a single event's details by its identifier.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).

Outputs:

- The requested Event object. (same as objects in Get Open Events Service service)

Usage / Interaction Rules:

- Clients must send a GET request to /api/events/{eventId}.
- Authentication is not required.
- On success, returns HTTP 200 OK with the event.
- On failure, returns an appropriate HTTP error if the event cannot be found.

Service Name: Extend Event Timer Service

Description: Allows an Admin to extend the duration of an active event by a given number of seconds.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).
- additionalTime (int) - Number of seconds to add to the event's duration.

Outputs:

- eventId (UUID) - Identifier of the event.
- duration (int) - The event's new total duration, in seconds.
- endDateTime (datetime) - The event's new computed end date/time.

Usage / Interaction Rules:

- Clients must send a PATCH request to /api/admin/events/{id}/extend with a JSON body containing additionalTime.
- The request must include authentication (JWT). Admin role required.
- On success, returns HTTP 200 OK with the ExtendTimerResponse payload.
- On failure, returns an appropriate HTTP error with a message.

Event Register

Service Name: Register For Event Service

Description: Registers the authenticated participant for an event, optionally supplying a registration key for private events and dietary information.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).
- regKey (string) - Registration passkey, required only for private events (in the request body).
- dietaryReq (string) - Optional dietary requirement.
- allergies (string) - Optional allergy information.
- currentUserId (UUID) - Identifier of the authenticated user (extracted from the security context).

Outputs:

- regId (UUID) - Unique identifier of the registration.
- eventId (UUID) - Event the user registered for.
- registeredAt (datetime) - Timestamp of registration.
- dietaryReq (string) - Dietary requirement submitted.
- allergies (string) - Allergy information submitted.

Usage / Interaction Rules:

- Clients must send a POST request to /api/events/{eventId}/registered with an optional JSON body containing regKey, dietaryReq and allergies.
- The request must include authentication (JWT).

- On success, returns HTTP 201 Created with the EventRegistrationResponse payload.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Get My Registrations Service

Description: Retrieves all event registrations belonging to the authenticated participant.

Inputs:

- currentUserId (UUID) - Identifier of the authenticated user (extracted from the security context).

Outputs:

- A list of EventRegistrationResponse objects (same structure as Register For Event Service).

Usage / Interaction Rules:

- Clients must send a GET request to /api/events/my-registrations.
- The request must include authentication (JWT).
- On success, returns HTTP 200 OK with the JSON array of registrations.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Get Event Participants Service

Description: Retrieves all participants of an event, including their team assignment and role.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).

Outputs:

- A list of EventParticipant objects, each containing:
- userId (UUID) - Identifier of the participant.
- fullName (string) - Participant's full name.
- email (string) - Participant's email address.
- teamId (UUID) - Identifier of the participant's team, if any.
- teamName (string) - Name of the participant's team, if any.
- teamRole (string) - The participant's role within the team (e.g. "LEADER", "MEMBER").
- joinedAt (datetime) - When the participant joined the team.

Usage / Interaction Rules:

- Clients must send a GET request to /api/admin/events/{id}/participants.
- The request must include authentication (JWT). Admin role required.
- On success, returns HTTP 200 OK with the JSON array of participants.
- On failure, returns an appropriate HTTP error with a message.

Forums

Service Name: Get Forum Posts Service

Description: Retrieves the list of forum posts for an event, visible to admins, participants, and superadmins.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).
- currentUserId (UUID) - Identifier of the authenticated user (extracted from the security context).

Outputs:

- A list of ForumPostSummary objects, each containing:
- postId (UUID) - Unique identifier of the post.
- title (string) - Post title.
- author (ForumAuthor) - userId (UUID), firstName (string), lastName (string), role (string).
- createdAt (datetime) - Timestamp of creation.
- replyCount (long) - Number of comments on the post.

Usage / Interaction Rules:

- Clients must send a GET request to /api/events/{eventId}/forum/posts.
- The request must include authentication (JWT). Admin, Participant or Superadmin role required.

- On success, returns HTTP 200 OK with the JSON array of posts.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Get Forum Post Service

Description: Retrieves a single forum post, including its full body and comments.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).
- postId (UUID) - Identifier of the post (in the URL path).
- currentUserId (UUID) - Identifier of the authenticated user (extracted from the security context).

Outputs:

- postId (UUID) - Unique identifier of the post.
- title (string) - Post title.
- body (string) - Post content.
- author (ForumAuthor) - userId (UUID), firstName (string), lastName (string), role (string).
- createdAt (datetime) - Timestamp of creation.
- comments (list) - A list of ForumComment objects, each containing:
- commentId (UUID) - Unique identifier of the comment.
- postId (UUID) - Post the comment belongs to.
- author (ForumAuthor) - userId (UUID), firstName (string), lastName (string), role (string).
- body (string) - Comment content.
- createdAt (datetime) - Timestamp of creation.

Usage / Interaction Rules:

- Clients must send a GET request to /api/events/{eventId}/forum/posts/{postId}.

- The request must include authentication (JWT). Admin, Participant or Superadmin role required.
- On success, returns HTTP 200 OK with the ForumPostDetailResponse payload.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Create Forum Post Service

Description: Allows a user to create a new forum post for an event.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).
- title (string) - Post title (required, max 255 characters).
- body (string) - Post content (required).
- currentUserId (UUID) - Identifier of the authenticated user (extracted from the security context).

Outputs:

- The created ForumPostDetail object. (as shown in Get Forum Post service)

Usage / Interaction Rules:

- Clients must send a POST request to /api/events/{eventId}/forum/posts with a JSON body containing title and body.
- The request must include authentication (JWT). Admin, Participant or Superadmin role required.
- On success, returns HTTP 201 Created with the created post.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Create Forum Comment Service

Description: Allows a user to comment on an existing forum post.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).
- postId (UUID) - Identifier of the post being commented on (in the URL path).
- body (string) - Comment content (required).
- currentUserId (UUID) - Identifier of the authenticated user (extracted from the security context).

Outputs:

- The created ForumComment object. (as shown in Get Forum Post service)

Usage / Interaction Rules:

- Clients must send a POST request to /api/events/{eventId}/forum/posts/{postId}/comments with a JSON body containing body.
- The request must include authentication (JWT). Admin, Participant or Superadmin role required.
- On success, returns HTTP 201 Created with the created comment.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Get Forum Permissions Service

Description: Returns the requesting user's forum permissions for the event (whether they can post, comment, or moderate).

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).
- currentUserId (UUID) - Identifier of the authenticated user (extracted from the security context).

Outputs:

- canCreatePost (boolean) - Whether the user can create a post.
- canComment (boolean) - Whether the user can comment.
- canModerate (boolean) - Whether the user can delete posts/comments.

Usage / Interaction Rules:

- Clients must send a GET request to /api/events/{eventId}/forum/permissions.
- The request must include authentication (JWT). Admin, Participant or Superadmin role required.
- On success, returns HTTP 200 OK with the ForumPermissionResponse payload.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Stream Forum Service

Description: Server-Sent Events stream that pushes forum updates (new posts, comments, deletions) in real time for a given event.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).

Outputs:

- Server-Sent Events (SSE) stream. Each event contains a forum update.

Usage / Interaction Rules:

- Clients must send a GET request to /api/events/{eventId}/forum/stream.
- Authentication is not required for this endpoint.
- The endpoint produces MediaType.TEXT_EVENT_STREAM_VALUE.
- On success, returns an SSE stream with forum update events.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Delete Forum Post Service

Description: Deletes a forum post. Available to the post's author and to moderators.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).
- postId (UUID) - Identifier of the post to delete (in the URL path).
- currentUserId (UUID) - Identifier of the authenticated user (extracted from the security context).

Outputs:

None (only HTTP status).

Usage / Interaction Rules:

- Clients must send a DELETE request to /api/events/{eventId}/forum/posts/{postId}.
- The request must include authentication (JWT). Admin, Participant or Superadmin role required.
- On success, returns HTTP 204 No Content.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Delete Forum Comment Service

Description: Deletes a forum comment. Available to the comment's author and to moderators.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).
- commentId (UUID) - Identifier of the comment to delete (in the URL path).
- currentUserId (UUID) - Identifier of the authenticated user (extracted from the security context).

Outputs:

None (only HTTP status).

Usage / Interaction Rules:

- Clients must send a DELETE request to /api/events/{eventId}/forum/comments/{commentId}.
- The request must include authentication (JWT). Admin, Participant or Superadmin role required.
- On success, returns HTTP 204 No Content.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Admin Delete Forum Post Service

Description: Admin-scoped variant that allows an Admin or Superadmin to delete any forum post for moderation purposes.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).
- postId (UUID) - Identifier of the post to delete (in the URL path).
- currentUserId (UUID) - Identifier of the authenticated admin (extracted from the security context).

Outputs:

None (only HTTP status).

Usage / Interaction Rules:

- Clients must send a DELETE request to /api/admin/events/{eventId}/forum/posts/{postId}.
- The request must include authentication (JWT). Admin or Superadmin role required.
- On success, returns HTTP 204 No Content.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Admin Delete Forum Comment Service

Description: Admin-scoped variant that allows an Admin or Superadmin to delete any forum comment for moderation purposes.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).
- commentId (UUID) - Identifier of the comment to delete (in the URL path).
- currentUserId (UUID) - Identifier of the authenticated admin (extracted from the security context).

Outputs:

None (only HTTP status).

Usage / Interaction Rules:

- Clients must send a DELETE request to /api/admin/events/{eventId}/forum/comments/{commentId}.
- The request must include authentication (JWT). Admin or Superadmin role required.
- On success, returns HTTP 204 No Content.
- On failure, returns an appropriate HTTP error with a message.

Superadmin

Service Name: Create Admin Service

Description: Allows a Superadmin to create a new Admin account for the platform.

Inputs:

- firstName (string) - First name of the new admin (2-100 characters).
- lastName (string) - Last name of the new admin (2-100 characters).
- email (string) - Email of the new admin.
- password (string) - Password for the new admin account (8-100 characters).

Outputs:

- token (string) - JWT token.
- userId (UUID) - ID of the newly created admin.
- firstName (string) - First name of the admin.
- lastName (string) - Last name of the admin.
- email (string) - Email of the admin.
- role (string) - The assigned role.

Usage / Interaction Rules:

- Clients must send a POST request to /api/superadmin/admin with a JSON body containing firstName, lastName, email and password.
- The request must include authentication (JWT). Superadmin role required.
- On success, returns HTTP 201 Created with the AuthResponse payload.
- On failure, returns an appropriate HTTP error with a message.

Service Name: Get Admins Service

Description: Retrieves the list of all Admin accounts on the platform.

Inputs:

- currentUserId (UUID) - Identifier of the authenticated superadmin (extracted from the security context).

Outputs:

- A list of Admin objects, each containing:
- userId (UUID) - Identifier of the admin.
- firstName (string) - First name of the admin.
- lastName (string) - Last name of the admin.
- email (string) - Email of the admin.
- status (string) - Account status.

Usage / Interaction Rules:

- Clients must send a GET request to /api/superadmin/admins.
- The request must include authentication (JWT). Superadmin role required.
- On success, returns HTTP 200 OK with the JSON array of admins.
- On failure, returns an appropriate HTTP error with a message.

Certificate

Service Name: Get Participation Certificate Service

Description: Generates and returns a PDF participation certificate for the authenticated user for a given event.

Inputs:

- eventId (UUID) - Identifier of the event (in the URL path).
- currentUserId (UUID) - Identifier of the authenticated user (extracted from the security context).

Outputs:

- A PDF file (binary), served inline with a generated filename of the form certificate-{firstname}.pdf.

Usage / Interaction Rules:

- Clients must send a GET request to /api/events/{eventId}/certificate.
- The request must include authentication (JWT).
- On success, returns HTTP 200 OK with Content-Type: application/pdf and a Content-Disposition: inline header.
- On failure, returns an appropriate HTTP error with a message.